PQC Secure-Code Reviewer Agent

Complete Architectural Anatomy & File-by-File Technical Guide

AGENTIC CYBER DEFENCE | NIST FIPS 203 (ML-KEM) | NIST FIPS 204 (ML-DSA) | ZERO-TRUST HITL GATE | SHA-256 HASH CHAINING

Project Summary for Collaborators: An autonomous yet strictly governed AI security tool designed to find and remediate quantum-vulnerable cryptographic algorithms (RSA, ECC, DES) in source code. It uses deterministic AST parsing, LLM Chain-of-Thought (CoT) reasoning, non-negotiable Human-in-the-Loop (HITL) approval, and produces a tamper-evident SHA-256 audit log.

1. Project Directory Anatomy

The codebase is organized into modular layers ensuring clear separation between the presentation tier, agent logic, detection tools, testbench, and DevSecOps pipelines:

```
hackathon-cybersecurity-agents/ ├── app.py # Streamlit interactive Web Dashboard (Port 8501) ├── pqc_config.yaml # Enterprise policy & target NIST algorithms ├── requirements.txt # Production Python dependency manifest ├── Dockerfile & docker-compose.yml # Hardened multi-stage container deployment | ├── pqc_agent/ # Core Agent Python Package | ├── __init__.py # Public exports & package initializer | ├── state.py # LangGraph AgentState TypedDict & Pydantic models | ├── tools.py # AST/Regex scanner & FIPS 203/204 patch generator | ├── prompts.py # System prompts & Chain-of-Thought templates | ├── graph.py # 5-node LangGraph pipeline & interrupt() HITL gate | ├── audit.py # Cryptographic SHA-256 recursive hash chain logger | └── cli.py # Headless DevSecOps CLI engine for CI/CD runs | ├── sample_files/ # Testbench: intentionally vulnerable code samples | ├── vulnerable_python.py # Python sample (RSA, ECDSA, DES, MD5, AES-128) | └── VulnerableJava.java # Java sample (RSA, ECDSA, DSA, DES, MD5, DH) | ├── .github/workflows/ # Automated DevSecOps CI/CD Pipelines | ├── ci.yml # Unit tests & LangGraph E2E verification | └── security-scan.yml # SAST (Bandit), SCA (pip-audit), PQC audit gate | ├── test_pqc_agent.py # Automated unit & tool integration test suite ├── test_graph_e2e.py # End-to-end LangGraph HITL interrupt & resume test └── audit_outputs/ # Generated tamper-evident audit logs (JSON & TXT)
```

2. The Core Agent Engine (`pqc_agent/`)

The primary intelligence and execution mechanisms reside within the `pqc_agent/` package:

File	Role & Responsibilities	Key Components Inside
`pqc_agent/state.py`	**State Schema & Data Contract** Defines the canonical TypedDict and Pydantic models flowing through every LangGraph node.	• `AgentState` (TypedDict) • `CryptoFinding` (Line, severity, conf) • `PQCPatch` (Diff, FIPS standard) • `AuditEntry` (Actor, action, timestamp)
`pqc_agent/tools.py`	**Scanning & Patch Synthesis Engine** Performs AST-based (Python) and regex (Java) detection; rewrites legacy calls to NIST ML-KEM/ML-DSA.	• `code_scanner_tool` (AST Visitor) • `pqc_patch_generator_tool` • `LEGACY_CRYPTO_DB` (Knowledge base) • Template synthesizers (liboqs / Bouncy Castle)
`pqc_agent/graph.py`	**LangGraph State Machine (The Brain)** Orchestrates the 5-node workflow. Pauses at human approval using native `interrupt()`.	• `node_ingest` (Validation & genesis hash) • `node_scan` (Deterministic scan) • `node_investigate` (LLM CoT reasoning) • `node_human_approval` (HITL Gate) • `node_finalise` (Seals audit trail)
`pqc_agent/prompts.py`	**LLM Cognitive Guidance** Prompts defining the agent's persona, rules, and structured investigation questions.	• `SYSTEM_PROMPT` (Persona & NIST rules) • `INVESTIGATION_PROMPT` (CoT & uncertainty) • `PATCH_SUMMARY_PROMPT` (FIPS notes)
`pqc_agent/audit.py`	**Cryptographic Provenance Engine** Creates tamper-evident logs using a recursive SHA-256 hash chain for non-repudiation.	• `_compute_chain_hash()` • `generate_audit_json()` • `generate_audit_txt()`
`pqc_agent/cli.py`	**Headless DevSecOps Scanner** CLI tool for CI/CD runners. Reads policy, scans files, generates patches, and returns exit codes.	• `scan_target()` recursive walker • Policy violation gate (Exit 0, 1, 2) • Automated JSON/TXT report export

3. Presentation, Testbench & DevSecOps Layer

File / Directory	Role & Responsibilities	Key Features
`app.py`	**Interactive Web Dashboard** The user-facing Streamlit application offering a clean cybersecurity control center.	• Dark glassmorphic theme (Port 8501) • Visual 5-phase execution tracker • Side-by-side code diff comparison • Approve/Reject HITL resume buttons • Instant JSON/TXT audit trail download
`pqc_config.yaml`	**Enterprise Policy Configuration** Defines policy failure thresholds, target NIST algorithms, and hybrid pairings.	• `fail_on_severity: CRITICAL` • `targets: ML-KEM-768, ML-DSA-65` • File inclusion/exclusion globs
`sample_files/`	**Vulnerable Code Testbench** Intentionally vulnerable Python and Java files used to validate detection and patching.	• `vulnerable_python.py` (21 findings) • `VulnerableJava.java` (12 findings) • Covers RSA, ECDSA, DSA, DES, MD5, DH
`test_pqc_agent.py`	**Unit & Integration Test Suite** Validates scanner detection, patch generation, hash chaining, and graph compilation.	• Automated assertion of 33+ findings • Verifies SHA-256 chain length & integrity • Verifies AST vs regex accuracy
`test_graph_e2e.py`	**E2E LangGraph State Machine Test** Proves the `interrupt()` pause and `Command(resume= ... )` unblocking lifecycle.	• Verifies graph halts at node 4 • Tests resume with human decision • Verifies final sealed audit trail
`.github/workflows/`	**Continuous Integration & Security** Automated GitHub Actions running quality and cryptographic gates on every commit.	• `ci.yml`: Matrix tests (Python 3.11, 3.12) • `security-scan.yml`: Bandit + pip-audit + PQC audit artifact upload
`Dockerfile` & `docker-compose.yml`	**Production Containerization** Hardened multi-stage container build adhering to CIS security benchmarks.	• Unprivileged user (`pqcuser`, UID 10001) • Dropped root capabilities • Native healthcheck (`/_stcore/health`) • Persistent volume for audit outputs

4. How the Files Interact (Runtime Workflow Sequences)

To explain the architecture to your team, walk them through the two primary execution workflows:

🌐 Workflow A: Interactive Web Review (`app.py`)

1. User uploads code or selects a sample in `app.py`.
2. `app.py` initiates the LangGraph state machine in `pqc_agent/graph.py`.
3. **Node 1 (Ingest):** Ingests file, generates genesis hash via `audit.py`.
4. **Node 2 (Scan):** Deterministically calls `tools.py` (AST visitor) to detect vulnerabilities.
5. **Node 3 (Investigate):** Calls `tools.py` for patches and queries LLM via `prompts.py`.
6. **Node 4 (HITL Gate):** LangGraph **pauses** via `interrupt()`.
7. `app.py` renders findings, CoT reasoning, and diffs.
8. Human clicks *Approve*; `app.py` calls `Command(resume= ... )`.
9. **Node 5 (Finalise):** Seals decision and writes SHA-256 audit log.

⚡ Workflow B: Headless CI/CD Pipeline (`cli.py`)

1. CI/CD runner executes:
   `python -m pqc_agent.cli --target src/`
2. `cli.py` loads policy rules from `pqc_config.yaml`.
3. `cli.py` recursively scans files using `tools.py` without needing web UI or LLM tokens.
4. If `--generate-patches` is passed, it generates NIST FIPS 203/204 replacement snippets.
5. `cli.py` invokes `audit.py` to save tamper-evident JSON/TXT logs.
6. `cli.py` checks violations against policy:
   - If findings ≥ CRITICAL: **Exit Code 1** (Build fails).
   - If clean: **Exit Code 0** (Build passes).

5. Quick Commands Cheatsheet for Team Demos

Launch Web Dashboard	`streamlit run app.py`	Opens interactive dark UI at http://localhost:8501
Run Headless CLI Scan	`python -m pqc_agent.cli --target sample_files/ --generate-patches`	Prints vulnerability table & generates PQC patches
Run Unit & Tool Tests	`python test_pqc_agent.py`	Tests AST scanner, patch generator & hash chaining
Run LangGraph E2E Test	`python test_graph_e2e.py`	Proves the interrupt pause & resume cycle
Launch with Docker	`docker compose up -d --build`	Starts containerized app on port 8501